

Project Proposal (Proposed Solution) Report

Date 06 July 2026

Team ID *****

Project Name StudyAI – AI Powered Smart Study Companion

Proposal summary: StudyAI removes the manual bottleneck between "raw study material" and "exam-ready prep." Students upload their own PDFs/DOCX/TXT and the platform automatically extracts content and generates summaries, flashcards, and quizzes using an LLM (Groq/Llama), cutting prep time and standardizing quality across subjects.

Project Overview

Objective Automatically convert a student's own uploaded study material into structured, exam-ready learning resources (summary, flashcards, quiz, revision plan), reducing manual prep time.

Scope Covers material upload and parsing, AI-generated summaries, flashcard and quiz generation, a study planner, and a dashboard to track progress across subjects.

Problem Statement

Description Students spend disproportionate time manually converting lecture notes/PDFs into revision-ready formats (summaries, flashcards, question banks), with no tool that automates this directly from their own material.

Impact Reduces exam prep time, improves consistency of revision material, and gives students a clearer, trackable path through their syllabus before exams.

Proposed Solution

Approach Extract text from uploaded files using PyMuPDF, pass it to the Groq API (Llama model) to generate summaries/flashcards/quiz questions, and serve results through a React frontend backed by a Flask API.

Key Features

- PDF/DOCX/TXT upload with UUID-based secure filename storage
- AI-generated summaries from extracted text

**Proposed
Solution**

- Auto-generated flashcards for active recall
- Auto-generated quizzes for self-testing
- Study planner to schedule revision across subjects
- Dashboard for tracking upload/progress history

Resource Requirements

Resource Type	Description	Specification/Allocation
Hardware		
Computing Resources	Backend runs standard CPU workload; no local GPU needed since inference is offloaded to Groq's hosted API	Standard dev machine / low-tier cloud VM (e.g., 2 vCPU)
Memory	RAM for Flask app + PDF parsing	4 GB minimum
Storage	Uploaded files + generated content	10–20 GB (scales with user uploads; move to cloud storage like S3 for production)
Software		
Frameworks	Frontend / Backend	React.js (with React Router) / Flask + Flask-CORS
Libraries	PDF processing, AI integration	PyMuPDF (fitz), Groq API client
Development Environment	IDE / VCS	VS Code, Git, GitHub
Data		
Data	Source, size, format	User-uploaded PDF/DOCX/TXT files (variable size); no fixed dataset — content is generated per-upload via LLM, not trained on a static dataset